

String Handling

String declaration (4 ways):

1. String s= "COMPUTER"
2. String s= new String("COMPUTER");
3. char ch[]={'C', 'O', 'M', 'P', 'U', 'T', 'E', 'R'};
String s= new String(ch);
4. char ch[] = {'C', 'O', 'M', 'P', 'U', 'T', 'E', 'R'};
String s = String.valueOf(ch);

COMPUTER

0 1 2 3 4 5 6 7

** There are other ways using StringBuilder and
StringBuffer (For now we can skip them as they are not included in ICSE Syllabus)

String functions (int returning functions):

Sl no.	Purpose	Function with example String s= "COMPUTER";	Range of values	Return type
1	To find length of a String	int len= s.length(); len = 8	>=0	int
2	To find index of first occurrence of a character. (returns -1 if character is not present)	int k= s.indexOf('P'); k= 3	>= -1	int
3.	To find index of a character at or after an index	String s2= "RACECAR" int k= s2.indexOf('A',2); k= 5	>= -1	int
4.	To find last index of a character	String s3= "RACECAR"; int k=s3.lastIndexOf('R'); k=6	>=-1	int
5.	To find index of first occurrence of a substring	int k= s.indexOf("PUTER"); k=3	>= -1	int
6.	To compare two Strings lexicographically Used for finding alphabetical order of two Strings int r= s1.compareTo(s2); r==0 → s1==s2 r < 0 → s1 < s2 r > 0 → s1 > s2	String s4= "APPLE", s5= "APLE"; int r= s4.compareTo(s5); r = 'P' - 'L' r= 80 - 76 r= 4 int r2= s5.compareTo(s4); r2= 'L' - 'P' r2= 76 - 80 r2= -4 int k= "APPLE".compareTo("APPLEJUICE"); k= -5 (length of left String - length of right String, as no mismatch is found) int m= "APPLE".compareTo("apple"); m= 'A' - 'a'; m= 65 - 97 m= -32 (‘A’ and ‘a’ are different for compareTo())	-∞ to ∞	int

7.	To compare two Strings lexicographically ignoring their case	<pre>int m= "APPLE".compareToIgnoreCase("apple"); m= 0 (length of left – length of right, as no mismatch is found) ('A' and 'a' are same for compareToIgnoreCase())</pre>	$-\infty$ to ∞	int

String function (char returning function)

	Purpose	Function with example String s= "COMPUTER";	Return type
8.	To get character of a position/index	<pre>char ch= s.charAt(4); ch= 'U'</pre>	char

String functions (String returning functions)

Sl no.	Purpose	Function with example String s= "COMPUTER";	Return type
9.	To get substring from an index	<pre>String ns= s.substring(3); ns= "PUTER"</pre>	String
10.	To get substring from an index(a,b) a is included, b is excluded	<pre>String ns= s.substring(2,6); ns= "MPUT"</pre>	String
11.	To replace a character by another	<pre>String ns= s.replace('U', 'O'); ns= "COMPOTER"</pre>	String
12.	To replace a substring by another	<pre>String ns= s.replace("TER", "tar"); ns= "COMPUtar"</pre>	String
13.	To remove leading and trailing spaces	<pre>String s= " Hello Java "; String ns= s.trim(); ns= "Hello Java"</pre>	String
14.	To join two Strings	<pre>String s1= "COMPUTER", s2= "APPLICATION"; String ns= s1.concat(s2); ns= "COMPUTERAPPLICATION"</pre>	String
15.	To convert a String to upper case	<pre>String ns= s.toUpperCase(); ns= "COMPUTER"</pre>	String
16.	To convert a String to lower case	<pre>String ns= s.toLowerCase(); ns= "computer"</pre>	String

String functions (boolean functions, can be used as conditions)

Sl No.	Purpose	Function with example String s= "COMPUTER";	Return type
17.	To check if two Strings are exactly equal or not (including case of each character)	<pre>String s1= "Apple", s2= "apple"; boolean b= s1.equals(s2); b= false Using directly in if() if(s1.equals(s2)){ //this block executes if s1 and s2 matches }</pre>	boolean

18.	To check if two Strings are equal or not (ignoring case of each character)	String s1= "Apple", s2= "apple"; boolean b= s1.equalsIgnoreCase(s2); b= true Using directly in if() if(s1.equalsIgnoreCase(s2)){ //this block executes if s1 and s2 matches ignoring cases } 	boolean
19.	To check if a String starts with a substring or not	boolean b= s1.startsWith("COM"); b=true Using directly with if() if(s1.startsWith(s2)){ //this block executes if s1 starts with s2 } 	boolean
20.	To check if a String ends with a substring or not	boolean b= s1.endsWith("TER"); b=true Using directly with if() if(s1.endsWith(s2)){ //this block executes if s1 ends with s2 } 	boolean
21.	To check if a String is empty or not	String s1= ""; boolean b=s1.isEmpty(); b=true String s2= " "; boolean b2= s2.isEmpty(); b2= false; Using directly with if() if(s1.isEmpty()){ //this block executes if s1 is empty } 	boolean
22.	To check if a String is blank or not	String s1= ""; boolean b=s1.isBlank(); b=true String s2= " "; boolean b2= s2.isBlank(); b2= true; Using directly with if() if(s1.isBlank()){ //this block executes if s1 is blank } 	boolean

** Trying to access a character beyond String's boundary causes runtime error:
StringIndexOutOfBoundsException

Examples of StringIndexOutOfBoundsException (Runtime Error) : String s= "COMPUTER";

```
char ch= s.charAt(8);  
char ch1= s.charAt(-1);  
char ch2= s.charAt(s.length())  
** remember, there is no character at s.length(), last character is located at s.length()-1
```

** String is immutable in JAVA – it means every time we try to modify a String, it creates a new String, an existing String **can not be modified**

** All String returning functions return a new modified String, it doesn't change the existing String
Example:

```
String s= "hello";  
String ns= s.toUpperCase();
```

```
System.out.println(ns); → HELLO
```

```
System.out.println(s); → hello
```

String is a Class

- Defined in java.lang package.
- Represents a **sequence of characters**.
- Treated as an **object**, not a primitive type.